

MANUAL TÉCNICO

Cronograma, Arquitectura, Tecnologías y Procedimientos

GYMPROYE

GymAR — Aplicación de Realidad Aumentada para Gimnasio

Elaborado por:

José Eduardo Villeda Tlecuítl

1. Introducción

El Manual Técnico describe cómo está diseñado y construido el proyecto GYMPROYE, el cual consiste en una aplicación móvil de Realidad Aumentada llamada GymAR para dispositivos Android. En este documento se explican la arquitectura del sistema, las tecnologías utilizadas, el cronograma de desarrollo y los pasos necesarios para construir y desplegar la aplicación.

Está dirigido principalmente a desarrolladores y personal técnico que necesiten entender cómo funciona internamente la app, cómo está organizado su código, qué herramientas y dependencias utiliza, y cómo pueden compilarla o implementarla.

Nota Técnica

Como nota técnica, GymAR es una aplicación nativa de Android desarrollada en Unity 2022 LTS, utilizando IL2CPP (que convierte el código a C++ compilado) para mejorar el rendimiento. El reconocimiento de marcadores se realiza con Vuforia Engine, mientras que el seguimiento del entorno se apoya en la cámara del dispositivo.

1.1 Objetivos del Documento

- Describir la arquitectura técnica del proyecto GymAR.
- Listar todas las tecnologías, SDKs y librerías utilizadas.
- Presentar el cronograma de actividades del proceso de desarrollo.
- Documentar la estructura interna del proyecto (archivos y directorios).
- Proveer instrucciones de compilación y despliegue del APK.

2. Cronograma de Actividades

El desarrollo del proyecto GymAR siguió una metodología iterativa dividida en 8 semanas de trabajo. A continuación, se presenta el cronograma de Gantt con las actividades principales realizadas durante el ciclo de desarrollo:

Actividad	S1	S2	S3	S4	S5	S6	S7	S8
1. Levantamiento de requerimientos	■							
2. Investigación de tecnologías (Unity/Vuforia/ARCore)	■	■						
3. Diseño de la arquitectura del proyecto		■						
4. Configuración del entorno de desarrollo		■	■					
5. Integración de SDK Vuforia en Unity			■					
6. Integración de ARCore			■	■				
7. Diseño y registro de marcadores (targets)				■				
8. Desarrollo de escenas Unity				■	■			
9. Creación de modelos/contenido 3D					■			
10. Implementación de lógica C# (scripts)					■	■		
11. Integración UI / menús						■		
12. Pruebas en dispositivo Android						■	■	
13. Corrección de errores y optimización							■	
14. Generación del APK de producción							■	■
15. Elaboración de documentación								■

2.1 Descripción de Fases

Fase 1 (S1–S2)	Se realizó la definición de los requerimientos del proyecto, junto con una investigación de los SDKs de Realidad Aumentada más adecuados. Además, se llevó a cabo la configuración del entorno de desarrollo en Unity para comenzar la implementación.
Fase 2 (S2–S3)	Se importó el modelo 3D del entrenador llamado Ch08_nonPBR, junto con sus texturas. Además, se realizó el rigging y la animación de las rutinas de ejercicio, incluyendo Bodybuilding y Stretching, utilizando la herramienta Mixamo.
Fase 3 (S4–S5)	Se llevó a cabo el desarrollo principal de la aplicación, incluyendo la creación de escenas, la integración de modelos 3D, la configuración de marcadores y la implementación de la lógica de negocio.
Fase 4 (S6–S7)	Se realizó la integración de la interfaz de usuario, junto con pruebas en dispositivos reales para verificar su funcionamiento. Además, se llevaron a cabo tareas de depuración y optimización para mejorar el rendimiento de la aplicación.
Fase 5 (S8)	Se generó el archivo APK final de la aplicación y se elaboró la documentación técnica correspondiente para su uso y mantenimiento.

3. Tecnologías Utilizadas

GymAR integra diversas tecnologías y frameworks especializados para ofrecer una experiencia de Realidad Aumentada fluida en dispositivos Android. A continuación, se detallan todas las herramientas involucradas:

Unity 2022 LTS	C# / IL2CPP Backend Nativo	Vuforia Engine SDK para RA
ARCore (Google) AR SDK Android	Android SDK API 24+	Gradle Plugin 7.4.2
Unity Animator Máquina de Estados	TextMeshPro Texto en Unity	PlayerPrefs Almacenamiento Local
Unity Timeline Animaciones	Mixamo Rigging y Animaciones	UnityWebRequest Comunicación HTTP

3.1 Unity Engine

Unity es el motor principal utilizado en el desarrollo de GymAR. Este se encarga del renderizado en 3D, la gestión de escenas, así como de los sistemas de física, animaciones y la interfaz de usuario.

La aplicación fue compilada utilizando IL2CPP, una tecnología que convierte el código escrito en C# a C++ nativo, lo que permite mejorar el rendimiento de la aplicación en dispositivos Android.

Motor	Unity 2022 LTS
Backend de Scripting	IL2CPP (C++ nativo)
Sistema de Renderizado	Universal Render Pipeline (URP)
Plataforma de Build	Android (ARM64 / arm64-v8a)
Orientación de pantalla	Portrait1080 x 1920
Fullscreen	Sí (androidStartInFullscreen=1)
Render fuera del SafeArea	Habilitado

3.2 Vuforia Engine

Vuforia Engine es el SDK de Realidad Aumentada utilizado para detectar superficies planas mediante la tecnología Ground Plane. Este sistema analiza en tiempo real la imagen capturada por la cámara, identifica patrones previamente registrados en su base de datos y calcula la posición y orientación del marcador. Con esta información, es posible colocar y fijar con precisión los objetos 3D en el entorno real.

SDK	Vuforia Engine
Integrado como	VuforiaScripts.dll + Vuforia.Unity.Engine.dll
Content Provider	com.vuforia.engine.app.VuforiaContentProvider
Tipo de tracking	Ground Plane (Detección de superficies horizontales)
Permisos que utiliza	Cámara, Internet
Licencia	Licencia de desarrollador Vuforia

3.3 ARCore (Google)

Google ARCore complementa a Vuforia Engine al proporcionar funciones de seguimiento del entorno. Entre estas se incluyen la detección de superficies planas, la estimación de la iluminación y el seguimiento del movimiento del dispositivo en el espacio 3D.

Para poder utilizar ARCore, es necesario que el dispositivo sea compatible y que tenga instalado Google Play Services for AR.

SDK	Google ARCore
Paquete de Play Store	Google Play Services for AR
Activity de instalación	com.google.ar.core.InstallActivity
Versión mínima APK	Especificada en meta-data de Manifest
Capacidades	Detección de planos, tracking, luz ambiental

3.4 Librerías Adicionales

Unity.InputSystem	Nuevo sistema de entrada de Unity para táctil y gestos.
Unity.TextMeshPro	Renderizado de texto de alta calidad en escenas Unity.
Unity.InputSystem	Nuevo sistema de entrada de Unity para táctil y gestos.
Unity.TextMeshPro	Renderizado de texto de alta calidad en escenas Unity.
UnityEngine.ARModule	Módulo de RA nativo de Unity integrado con ARCore.
UnityEngine.UI	Módulo base para la gestión de Canvas, botones y eventos de interfaz.

4. Estructura del Proyecto

La aplicación compilada se distribuye como un archivo APK. A continuación se describe la estructura interna del APK generado por Unity y su significado técnico:

4.1 Estructura del APK

Ruta / Archivo	Descripción
<code>GYMPROYE.apk</code>	Paquete de distribución Android
<code>AndroidManifest.xml</code>	Declaración de componentes, permisos y configuración Android
<code>classes.dex</code>	Bytecode Dalvik: clases Java/Kotlin del wrapper Android
<code>resources.arsc</code>	Recursos compilados de Android
<code>res/</code>	Recursos de la app: imágenes PNG, XML de layouts
<code>lib/arm64-v8a/</code>	Librerías nativas compiladas para arquitectura ARM64
<code>libil2cpp.so</code>	Runtime IL2CPP: toda la lógica C# compilada a C++ nativo
<code>libunity.so</code>	Motor Unity: renderizado, física, audio, input
<code>SDK Vuforia</code>	Tracking y detección de superficies planas
<code>libVuforiaUnityPlayer.so</code>	Puente Unity–Vuforia
<code>libarccore_sdk_c.so</code>	SDK ARCore en C nativo
<code>libarccore_sdk_jni.so</code>	Bridge JNI de ARCore para Java/Unity
<code>libUnityDriver.so</code>	Driver de Android para Unity Player
<code>libmain.so</code>	Entry point nativo de la aplicación Unity
<code>assets/bin/Data/</code>	Assets de Unity empaquetados
<code>data.unity3d</code>	Bundle principal: escenas, texturas, materiales, prefabs
<code>unity default resources</code>	Recursos por defecto del motor Unity
<code>Managed/Metadata/</code>	Metadatos de tipos .NET para IL2CPP en runtime
<code>ScriptingAssemblies.json</code>	Lista de ensamblados .NET cargados en runtime
<code>RuntimeInitializeOnLoads.json</code>	Métodos ejecutados al inicializar la app
<code>boot.config</code>	Configuración de arranque: HDR, SafeArea, GUID del build
<code>META-INF/</code>	Firmas y metadatos del APK.

4.2 Estructura del Proyecto Unity (Fuente)

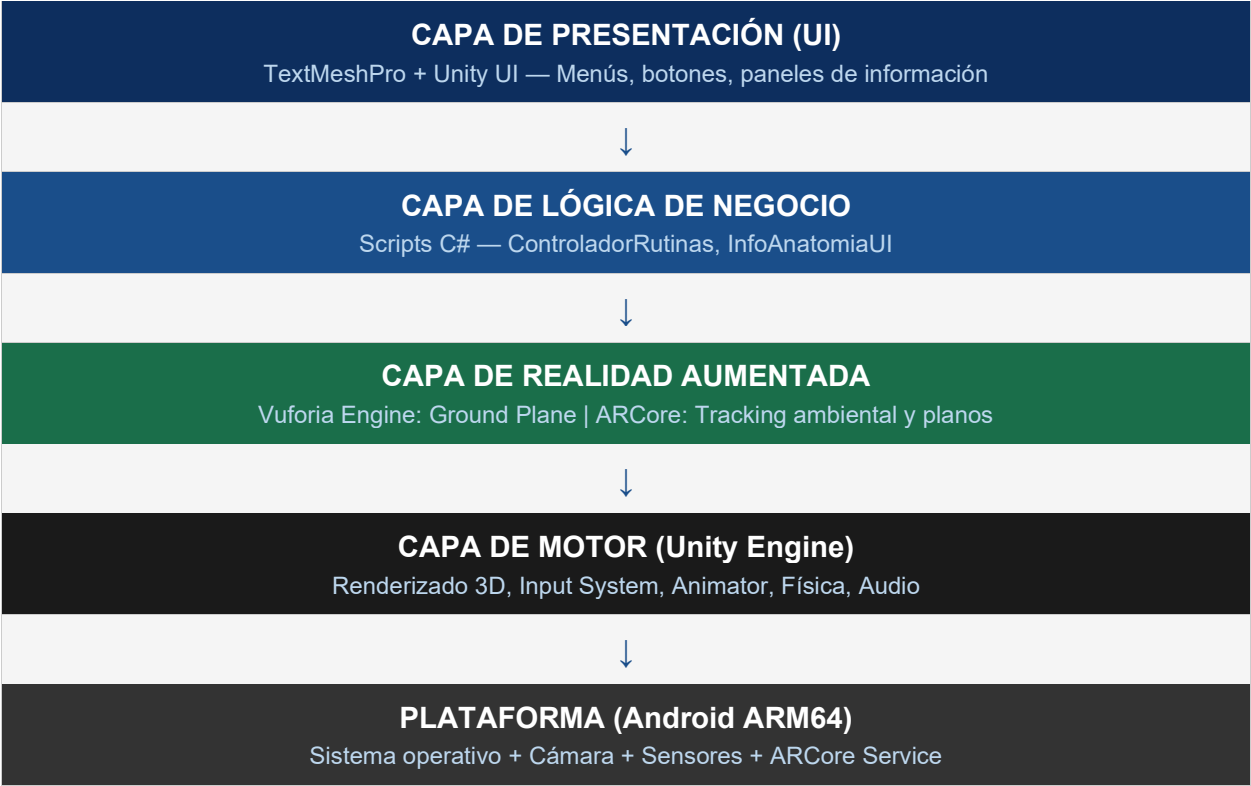
En el entorno de desarrollo de Unity, los archivos del proyecto se organizaron siguiendo la estructura estándar del motor, concentrando todos los elementos principales dentro del directorio de recursos:

GymAR/ Carpeta raíz del proyecto

- **Assets/** Contiene todos los recursos lógicos y visuales de la aplicación
 - **Scenes/:** Directorio con la escena principal donde se integró la interfaz y la cámara de AR.
 - **Scripts/:** Código fuente desarrollado en C#.
 - **ControladorRutinas.cs:** Script central para el control de la máquina de estados, transiciones de animación y activación de props 3D.
 - **InfoAnatomiaUI.cs:** Script dedicado a la inyección y actualización de textos descriptivos en el Canvas.
 - **Prefabs/:** Objetos prefabricados listos para instanciar (incluyendo las mancuernas ajustadas para el agarre del modelo).
 - **Models/:** Archivos 3D importados.
 - **Materials / Textures/:** Materiales, shaders y texturas utilizadas para la ropa del modelo y los botones de la interfaz gráfica.
 - **Plugins/:** Dependencias y SDKs externos importados al proyecto.
 - **Resources/:** Archivos empaquetados que la aplicación requiere cargar en memoria en tiempo de ejecución.
- **Packages/:** Módulos oficiales instalados a través del Unity Package Manager.
- **ProjectSettings/:** Configuraciones generales de compilación, físicas y calidad del proyecto.
- **Library/:** Archivos temporales y caché generados localmente por el motor.

5. Arquitectura del Sistema

GymAR sigue una arquitectura de capas adaptada al paradigma de Unity, donde cada capa tiene responsabilidades bien definidas:



5.1 Flujo de Datos AR

El flujo de procesamiento de Realidad Aumentada en GymAR sigue el siguiente ciclo por cada frame renderizado:

1. Captura	La cámara del dispositivo captura el frame de video en tiempo real.
2. Análisis	Vuforia procesa el frame buscando superficies planas Ground Plane horizontales en el entorno.
3. Detección	Al detectar una superficie adecuada, Vuforia establece un ancla espacial en esa posición.
4. Transformación	Unity actualiza la transformada del contenido AR para mantenerlo anclado a la superficie detectada.
5. Renderizado	Unity renderiza el frame combinando el video de cámara con el contenido 3D superpuesto.
6. Presentación	El frame final se muestra en la pantalla del dispositivo a 30/60 FPS.

6. Procedimientos de Compilación y Despliegue

6.1 Requerimientos del Entorno de Desarrollo

Unity Hub	Versión 3.x o superior
Unity Editor	2022 LTS con módulo Android Build Support
Android SDK	API Level 24 o superior
Android NDK	r23b o superior para IL2CPP
JDK	OpenJDK 11 incluido con Unity
Gradle Plugin	7.4.2 configurado en el proyecto
Vuforia Developer Portal	Cuenta y licencia activa en developer.vuforia.com
ARCore	Dispositivo físico Android compatible con ARCore
USB Debugging	Habilitado en el dispositivo para pruebas

6.2 Proceso de Compilación del APK

Para generar el APK de distribución desde el proyecto Unity, siga estos pasos:

Paso 1: Configurar Build Settings

- En Unity, ve al menú superior y selecciona la opción File > Build Settings.
 - Dentro de la lista de plataformas, elige Android y haz clic en Switch Platform. El motor tardará unos momentos en adaptar e importar los recursos al formato móvil.
 - Finalmente, haz clic en Add Open Scenes para asegurarte de que la escena principal, llamada *SampleScene*, quede incluida en la lista de compilación.

Paso 2: Configurar Player Settings

- Dentro de la misma ventana de compilación en Unity, accede a Player Settings.
 - Ahí debes configurar los datos básicos de la aplicación: el Company Name (por ejemplo, Jose), el Product Name (GymAR) y el Package Name con el formato *com.Jose.GymAR*.
 - En la sección de compatibilidad, establece el Minimum API Level en Android 7.0 y el Target API Level en Android 13 o superior.
 - Para optimizar el rendimiento, cambia el Scripting Backend de Mono a IL2CPP y, en Target Architectures, selecciona únicamente ARM64, desmarcando las demás opciones. Esto permitirá que el código se convierta a C++ nativo.
 - Por último, configura el Write Permission en External para asegurar que el sistema de guardado funcione correctamente.

Paso 3: Configurar Vuforia

- En Unity, ve al menú Window > Vuforia Configuration.
 - Dentro de esta sección, ingresa tu clave válida en el campo App License Key.
 - Luego, en Device Tracker, activa la opción Track Device Pose, ya que es necesaria para que la detección de superficies funcione correctamente.
 - Finalmente, verifica que Camera Device esté configurado como Default, lo que asegura el uso de la cámara trasera del dispositivo.

Paso 4: Generar el APK

- Regresa a la ventana de Build Settings en Unity.
 - De forma opcional, puedes activar la casilla Development Build si deseas realizar pruebas de rendimiento o profiling; sin embargo, esta opción debe estar desactivada para la versión final de producción.
 - Después, haz clic en Build o en Build and Run si tienes un dispositivo Android conectado por USB con la depuración activada.
 - Por último, selecciona la carpeta de destino en tu equipo. Unity procesará las librerías nativas y generará el archivo ejecutable GYMPROYE.apk, el cual estará listo para su instalación y distribución.

Tiempo de Compilación

La primera compilación con IL2CPP puede tardar entre 15 y 30 minutos dependiendo del hardware del equipo de desarrollo. Las compilaciones subsiguientes son más rápidas gracias al caché incremental de IL2CPP.

6.3 Instalación en Dispositivo

```
# Via ADB (Android Debug Bridge):
adb install GYMPROYE.apk

# Si el dispositivo ya tiene una versión instalada:
adb install -r GYMPROYE.apk

# Verificar instalación:
adb shell pm list packages | grep Jose.GymAR
```

7. Permisos y Configuración del Manifest

El archivo AndroidManifest.xml declara la configuración crítica de la aplicación. A continuación se detallan los permisos y configuraciones más relevantes extraídos del manifest de GymAR:

Package	com.Jose.GymAR
Activity principal	com.unity3d.player.UnityPlayerActivity
Orientación	portrait (Vertical)
Launch Mode	singleTask
Fullscreen	androidStartInFullscreen = true
Render SafeArea	androidRenderOutsideSafeArea = true
extractNativeLibs	true
glEsVersion	0x00030000
installLocation	auto

8. Control de Calidad y Pruebas

8.1 Estrategia de Pruebas

Pruebas Unitarias	Validación de scripts C# individuales (lógica de datos y UI).
Pruebas de Integración	Verificación de la comunicación entre Vuforia, ARCore y Unity.
Pruebas en Dispositivo	Ejecución en hardware Android real con distintas versiones del SO.
Pruebas de Rendimiento	Medición de FPS, uso de memoria y temperatura del dispositivo.
Pruebas de Usabilidad	Validación del flujo de usuario con personas reales.

8.2 Criterios de Aceptación

- La aplicación debe iniciar correctamente en dispositivos ARM64 con Android 7.0+.
- Vuforia debe detectar una superficie plana (Ground Plane) adecuada en menos de 2 segundos bajo iluminación normal.
- El contenido AR debe mantenerse anclado a la superficie detectada con una desviación menor a 5mm visuales.
- La aplicación debe mantener un mínimo de 25 FPS en dispositivos de gama media.
- Los permisos de cámara e Internet deben solicitarse correctamente en primera ejecución.
- La aplicación no debe cerrarse inesperadamente durante uso normal de 10 minutos.

10. Referencias

Unity Documentation	https://docs.unity3d.com/Manual/android-GettingStarted.html
Vuforia Developer Portal	https://developer.vuforia.com/
ARCore Documentation	https://developers.google.com/ar
Android Developer Docs	https://developer.android.com/docs
Unity IL2CPP	https://docs.unity3d.com/Manual/IL2CPP.html
Gradle Plugin Android	https://developer.android.com/build/releases/gradle-plugin
Vuforia Unity SDK	https://library.vuforia.com/unity/vuforia-engine-package-unity

LINK DEL GITHUB DEL PROYECTO:

https://github.com/Eduardovilleda/Proyecto_AR_VR.git